

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.12.20	完成日期	2024.12.20

第一题

1. 实验题目

最大生成森林

2. 实验目的

对连通图进行遍历, 过程中所经过的边和顶点的组合可看做是一棵普通树, 通常称为生成树。

而连通图的所有生成树中边权重和最大的那棵生成树即为最大生成树。

对于非连通图而言, 多个连通分量对应的多棵生成树就构成了整个非连通图的生成森林。

而最大生成森林的定义就是在非连通图中, 所有连通分量对应的最大生成树之和。

因此我们给定一个无向图 G , 它可能是一个非连通图的, 你的任务是找到最大生成森林。

3. 算法设计

1. 通过 Kruskal 算法和并查集实现了一个求解最大生成树的解决方案。

2. 核心思想是: ①按边的权值从大到小排序, 将 Kruskal 的求最小生成树的算法转变成求最大生成树; ②利用并查集判断是否形成环, 确保生成树的结构; ③挑选权值大的边, 构建最大生成树, 最终返回树的权值和。

4. 程序运行与测试

1. 带权边的结构体

```
struct edge
{
    int x;
    int y;
    int val;
    edge(int x,int y,int val):x(x),y(y),val(val)
    {}
};
```

2.用并查集来辅助判断有无环

```
vector<int> p(1000); //并查集
int Find(int x) {
    return p[x] == x ? x : p[x] = Find(p[x]);
}
```

3.Kruscal 算法

```
int Kruscal(int n,vector<edge> v)
{
    for (int i = 0; i < n; i++)
        p[i] = i; //初始化并查集
    sort(v.begin(), v.end(), cmp); //对边进行排序 贪心算法取最大的边
    int ans = 0;
    for (int i = 0; i < v.size(); i++)
    {
        int point1 = Find(v[i].x);
        int point2 = Find(v[i].y);
        if (point1 != point2) //表示二者不相连 加入该边不会形成环
        {
            p[point2] = point1;
            ans += v[i].val;
        }
    }
    return ans;
}
```

4.主函数

```

int main()
{
    int case_num;
    cin>>case_num;
    while(case_num--)
    {
        int n,m;//n顶点数 m边数
        cin>>n>>m;
        vector<edge> v;
        while(m--)
        {
            int x,y,d;//x与y的无向边 代价为d>0
            cin>>x>>y>>d;
            v.push_back(edge(x,y,d));
        }
        cout<<Kruscal(n,v)<<endl;
    }

    return 0;
}

```

5. 实验总结与心得

1. 实验题重点要掌握 Kruscal 算法，以及利用并查集来判断两个顶点之间是否连通
2. 对于此类带权图，要用带权边类型的数组来存储，带权边包括两端顶点和权值
3. 将题目求最大生成森林转化为最小生成树从大到小排序的解法
4. Kruscal 算法：①每次找带权边集中的最大权值边 ②判断加入改变是否会连通，使用并查集判断 ③不会则加入该边，并且总权重加上该边
5. 熟悉并查集的使用，在这里巧妙的用以判断是否联通，并且使用精简高效

第二题

1. 实验题目

Highways

2. 实验目的

The island nation of Flatopia is perfectly flat. Unfortunately, Flatopia has no public highways. So the traffic is difficult in Flatopia. The Flatopian government is aware of this problem. They're planning to build some highways

so that it will be possible to drive between any pair of towns without leaving the highway system.

Flatopian towns are numbered from 1 to N. Each highway connects exactly two towns. All highways follow straight lines. All highways can be used in both directions. Highways can freely cross each other, but a driver can only switch between highways at a town that is located at the end of both highways.

The Flatopian government wants to minimize the length of the longest highway to be built. However, they want to guarantee that every town is highway-reachable from every other town.

3. 算法设计

方法一：Kruskal 算法

1. 创建保存路线的矩阵图、保存父结点的数组、保存起点的数组、保存终点的数组、权重
2. 输入，在 main 函数将信息输入，初始化上面的一维数组和二维数组
- 3.Kruskal 算法：①初始化并查集和边按照权重排序的数组②对边按照权重进行排序③按照权重从低到高，每次取出权重最小的边④查找该边的起点和终点⑤如果两个端点不在同一集合则合并他们，并更新最大边权重

方法二：Prim 算法

1. 创建保存路线的矩阵图，并在 main 函数实现初始化
- 2.Prim 算法：①创建已经记录的结点 V_ok,未记录的结点 V_no 集合并进行初始化 ②循环，在已选择的节点里选择一个作起点，在未选择的节点里选择一个作终点，不断比较出目前权重最小的边 ③记录该边及其权重，更新 V_ok 集合和 V_no 集合，选择 set 容器更新时更高效，并更新最大权重边 maxn ④当 V_ok 的集合结点数收集了 N 个时退出

4. 程序运行与测试

方法一：Kruskal 算法

1. main 函数

```

int main()
{
    cin >> T;
    while (T--) {
        cin >> N; //读取每个测试用例的节点数量

        //将所有的边存储到start, end, value数组中
        int cou = 0;
        for (int i = 1; i<=N; i++)
        {
            for (int j = 1; j <=N; j++)
            {
                start[cou] = i;
                ending[cou] = j;
                cin >> value[cou]; //存储边的起点、终点和权重
                cou++;
                //cin >> map[i][j];
            }
        }
        cout << Prim() << endl; //计算并输出每个测试用例的结果
    }
    return 0;
}

```

2. 创建的数组和变量

```

int T, N;

int father[250010], start[250010], ending[250010], value[250010], sot[250010], map
[501][501];
//father是并查集的父亲点数组，用于记录每个结点的父节点
//start, end, value分别存储边的起点、终点和权重，用于Kruskal算法
//比较函数 用于排序边 按照边的权重排序

```

3. 并查集、比较递增排序的函数

```

bool cmp(const int i, const int j) {
    return value[i] < value[j];
}
//并查集查找函数
int u_find(int x) {
    return father[x] == x ? x : father[x] = u_find(father[x]);
}

```

4. Kruskal 算法

//Kruskal算法计算最小生成树

```
int Kruskal() {
    int maxn = 0; //记录最小生成树中最大边的权重
    for (int i = 0; i < N * N; i++) father[i] = i; //初始化并查集
    for (int i = 0; i < N * N; i++) sot[i] = i; //初始化边的排序数组

    sort(sot, sot + N * N, cmp); //对所有边按权重升序排序
    for (int i = 0; i < N * N; i++) {
        int e = sot[i]; //取出第i条边
        int x = u_find(start[e]), y = u_find(ending[e]); //查找边的两个端点的根节点
        if (x != y) { //如果两个端点不在同一个集合中，说明不会形成环
            father[y] = x; //合并两个集合
            maxn = max(maxn, value[e]); //更新最小生成树中的最大边权重
        }
    }
    return maxn; //返回最小生成树中最大边的权重
}
```

方法二：Prim 算法

1.main 函数

```
int main()
{
    cin >> T;
    while (T--) {
        cin >> N; //读取每个测试用例的节点数量

        //将所有的边存储到start, end, value数组中
        int cou = 0;
        for (int i = 1; i <= N; i++)
        {
            for (int j = 1; j <= N; j++)
            {
                //start[cou] = i;
                //ending[cou] = j;
                //cin >> value[cou]; //存储边的起点、终点和权重
                //cou++;
                cin >> map[i][j];
            }
        }
        cout << Prim() << endl; //计算并输出每个测试用例的结果
    }
    return 0;
}
```

3. Prim 算法

```

int Prim() {
    int maxn = 0; //记录生成树中最长边的权重
    set<int> V_ok;
    set<int> V_no;

    for (int i = 1; i <= N; i++)
        V_no.insert(i);
    V_ok.insert(1);
    V_no.erase(1);

    // 优先队列的构建
    while (V_ok.size() < N) {
        int min_weight = INT_MAX; //当前最小边的权重
        int min_edge = 0; //最小边的终点

        //遍历所有已选顶点
        for (auto it = V_ok.begin(); it != V_ok.end(); ++it)
        {
            int u = *it;
            //遍历所有未选顶点
            for (auto jt = V_no.begin(); jt != V_no.end(); ++jt)
            {
                int v = *jt;
                int weight = map[u][v];

                //如果该边的权重小于当前最小值
                if (weight < min_weight)
                {
                    min_weight = weight;
                    min_edge = v;
                }
            }
        }

        //更新V_ok和V_no集合
        V_ok.insert(min_edge);
        V_no.erase(min_edge);
        maxn = max(maxn, min_weight); //更新生成树中的最大边
    }

    return maxn;
}

```

5. 实验总结与心得

1. 解决最小生成树类型题目有两种，一个是 Kruskal 算法，一个是 Prim 算法
2. 两种算法都能解决问题，在不同的问题上操作的难易程度不同
3. Kruskal 算法：①初始化并查集和边按照权重排序的数组②对边按照权重进行排序③按照权重从低到高，每次取出权重最小的边；Prim 算法：①创建已经记录的结点 V_ok,未记录的结点 V_no 集合并进行初始化 ②循环，在已选择的

节点里选择一个作起点，在未选择的结点里选择一个作终点，不断比较出目前权重最小的边

4.做题还要能够判断出题目是使用最小生成树解决的题目，然后选择一种算法进行求解